

Deep Matching and Retrieval of Indic Script Documents

Pappu Meghana (S20230030400)

Introduction

- Indic Scripts are structurally complex writing systems
- Fundamental unit has a multi-component structure.
- Visual unit is not same as Unicode Representation.
- Many-to-many mapping between glyphs and Unicode sequences.
- Encoding is compositional rather than linear.
- Traditional string matching algorithms like Levenshtein edit distance capture syntactical similarity.
- Need representation-aware approaches for matching and retrieval of Indic text.

Problem Statement

- **Task:** Matching and retrieval of Indic string pairs
- **Inputs:** Two strings x_1 and x_2 in Unicode sequence form
- Learn distance function: $y = f(x_1, x_2)$
- Supervision using Levenshtein Edit Distance (LED)
- **Goal:** capture structural and encoding-level similarity
- Avoid explicit alignment at inference time

Scope:

- Script focus: **Telugu** (representative Indic script)
- Matching unit: **Word**-level similarity
- Controlled dataset with clean text

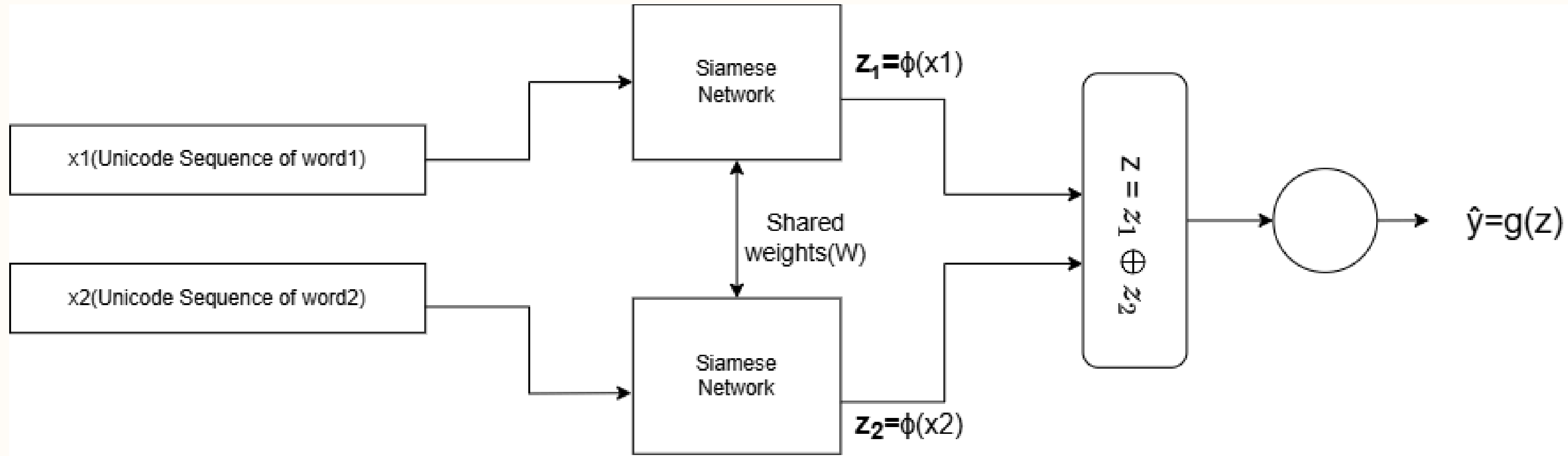
Motivation

- Encoded sequence length differs from visual word length
- Roman scripts: one-to-one mapping
- Indic scripts: akshara $\rightarrow$ multiple Unicode symbols
- Edit distance uses dynamic programming alignment
- Time complexity: $O(|x_1| \cdot |x_2|)$
- Computationally expensive for complex sequences
- Treats symbols as flat units (no structural awareness)
- Hence, need learning-based neural model to learn structure-aware similarity for Indic text

Literature Review

- Algorithmic Roots: Navarro (2001) established the limitations of alignment-based matching in quadratic time.
- Siamese Architectures: Bromley et al. (1993) pioneered shared-weight encoders for signature verification, now adapted for string similarity.
- Indic Context: Krishnan et al. (2015) highlighted that character-level methods fail for Indic scripts due to many-to-many mappings between glyphs and encodings.
- Metric Learning: Bellet et al. (2015) demonstrated how to learn embedding spaces where distances represent structural relationships.

Methodology



Unicode-Level Siamese Distance Learning

- Input Representation: Words are treated as sequences of Unicode code points $u_1, u_2, \dots, u_T$
- Vocabulary for One-Hot Encoding ($\mathcal{U}$): Telugu Unicode block (0x0C00–0x0C7F)
- Size of Unicode vocabulary ≈ 128 symbols
- Each Unicode symbol $\rightarrow$ One-hot vector
- One-hot vectors map to dense embeddings

$$e_i = E[u_i] \quad E \in \mathbb{R}^{|\mathcal{U}| \times d}$$

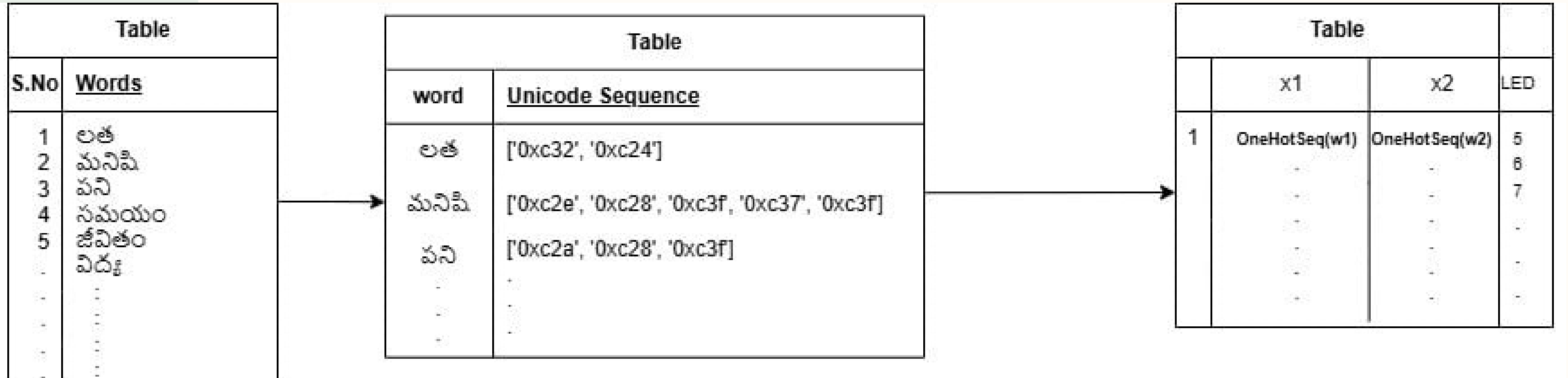
The Design Flow:

- Two inputs: x_1, x_2 (Unicode sequences)
- Shared encoder $\phi(\cdot)$
- $z_1 = \phi(x_1), z_2 = \phi(x_2)$
- Concatenation: $z = z_1 \oplus z_2$
- $g(\cdot)$: A function which outputs the predicted Edit Distance
- $\hat{y} = g(z)$
- Loss Function: Regression-based Mean Squared Error (MSE) to align predicted distance $\hat{y}$ with ground-truth LED.

Current Work

Data Collection and Preprocessing:

- **Word Vocabulary:** 500 clean Telugu words sampled for structural diversity.
- **Unicode Vocabulary:** Fixed Unicode vocabulary (0x0C00–0x0C7F, 128 code points) for one-hot representation.
- Data Pairs: $\binom{500}{2}$ possible pairs; Created a subset of 500 labeled pairs (x_1, x_2, y) .
- Ground Truth Labels: LED computed for each pair for distance learning.

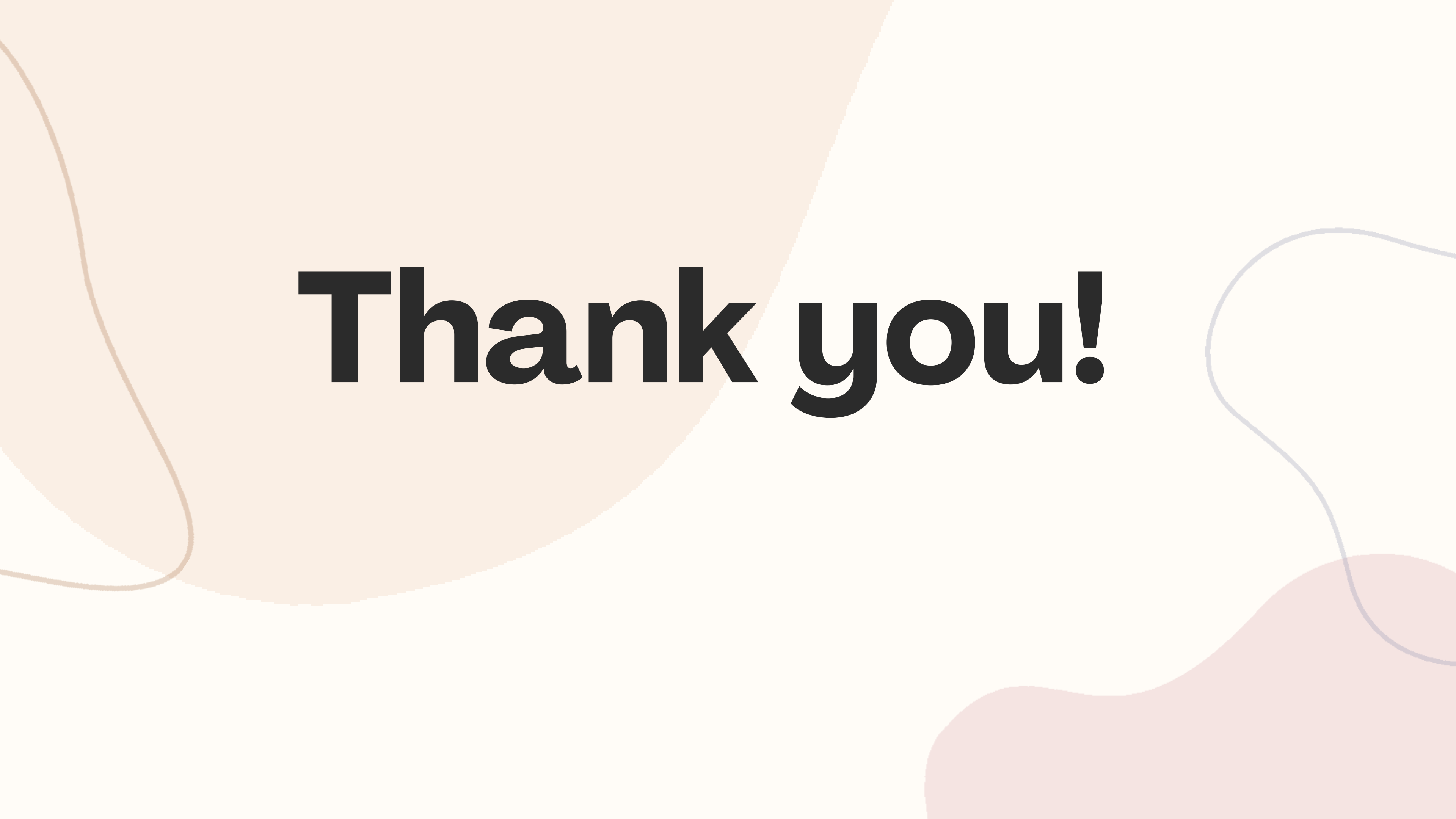


Future Plan and Timeline

- Learn compact embedding representations ($\text{dim} < 128$)
- Train regression-based Siamese DL model
- Performance evaluation and analysis
- Intuitive glyph-level input representation of words
- Extending this idea to other Indic Scripts using Transfer Learning

References

- Bellet et al., 2015 — Metric Learning for Structured Data
- Belinkov & Bisk, 2018 — Neural Models and Sequence Noise
- Bromley et al., 1993 — Siamese Neural Networks
- Graves, 2009 — Sequence Modeling with RNNs
- Krishnan et al., 2015 — Indic Document Word Spotting
- Navarro, 2001 — Approximate String Matching (Edit Distance)



Thank you!